CSL7640 : ASSIGNMENT-III: Machine Translation

B22CS001, B22CS093, B22BB016

31st March 2025

1 Introduction

Machine Translation (MT) is a crucial subfield of Natural Language Processing (NLP) that focuses on automatically translating text from one language to another. In this assignment, we develop a `Neural Machine Translation (NMT)` system to convert English sentences into Hindi using a sequence-to-sequence learning framework. Our approach involves two primary architectures: `LSTM-based Encoder-Decoder Model` – A traditional sequence-to-sequence model leveraging Long Short-Term Memory (LSTM) networks for both encoding and decoding. `Transformer Model` – A self-attention-based architecture designed to capture long-range dependencies more effectively than RNN-based models. Additionally, as part of our bonus experiment, we propose a `hybrid Transformer + BiLSTM model` to leverage the strengths of both architectures. The BiLSTM captures sequential dependencies, while the Transformer enhances contextual representation using self-attention. To train our models, we use pre-trained word embeddings such as `Word2Vec` or `GloVe`, ensuring meaningful word representations in a high-dimensional space. The dataset consists of parallel English-Hindi sentence pairs, with preprocessing steps like tokenization and special tokens `<sos>` and `<eos>` to mark sentence boundaries. For evaluation, we employ the **BLEU** score, a standard metric for assessing translation quality based on n-gram overlap with reference sentences. We also conduct fine-tuning experiments, ablation studies, and comparative analysis to determine the effectiveness of different architectures. Through this study, we aim to identify the strengths and limitations of each approach and analyze how our hybrid model improves translation performance.

2 Dataset Preparation

The **NMTDataset** class is responsible for loading, tokenizing, and preparing English-Hindi sentence pairs for training and evaluation. It extends PyTorch's `Dataset` class, ensuring efficient data processing. The class reads aligned English and Hindi sentences from files and tokenizes them by adding `<sos>` (start-of-sequence) and `<eos>` (end-of-sequence) tokens. Sentences are truncated to a maximum length of 50 to prevent memory issues. If vocabularies are not provided, the class constructs them by assigning unique indices to each word, including special tokens like `<pad>` (padding) and `<unk>` (unknown words). Additionally, embeddings are initialized using **Xavier initialization** if pre-trained `Word2Vec` or `GloVe` embeddings are unavailable.

2.1 Efficient Batching with `collate_fn`

To ensure efficient batching, the `collate_fn` function is used. It sorts sentences by length in descending order, reducing unnecessary padding and improving training efficiency. It also pads shorter sequences with the `<pad>` token and converts them into tensors, making them compatible for model input.

2.2 Key Features of Data Processing

- `Efficient Data Handling:` Loads and processes aligned English-Hindi sentence pairs.
- `Vocabulary Generation:` Creates word-to-index mappings for both languages, including special tokens.

- `Embedding Initialization:` Uses pre-trained embeddings if available or initializes them randomly.
- `Batched Processing:` The `collate_fn` function ensures sequences are sorted, padded, and converted into tensors for optimized training.

3 LSTM-Based-SeqToSeq Model Architecture

The **LSTM-based Sequence-to-Sequence Model** consists of an `LSTMEncoder`, `LSTMDecoder`, and a `LSTMSeq2SeqModel` that orchestrates translation.

3.1 LSTMEncoder

The **LSTMEncoder** embeds input tokens and processes sequences using a **bidirectional LSTM**, capturing contextual information from both directions. The final hidden states from both directions are concatenated and transformed via a **linear layer** before being passed to the decoder.

3.2 LSTMDecoder

The **LSTMDecoder** uses the encoder's final states as its initial hidden states and generates output tokens one by one. It applies **teacher forcing** during training and outputs probability distributions over the vocabulary.

3.3 LSTMSeq2SeqModel

The **LSTMSeq2SeqModel** combines the encoder and decoder to handle the full translation process. It manages hidden state transfer, ensuring proper initialization of the decoder's hidden states. The model iteratively generates translated sequences by feeding the decoder's output back as input in an **autoregressive** manner.

3.4 Conclusion

This architecture enables effective sequence-to-sequence translation with **LSTMs**, capturing long-range dependencies for robust performance.

4 Transformer Architecture

The **Transformer Model** is a groundbreaking architecture introduced in the paper *"Attention is All You Need"* and is widely used for tasks like machine translation. Unlike traditional models like RNNs and LSTMs, the Transformer relies on a self-attention mechanism, allowing it to process the entire input sequence simultaneously, making it highly efficient and capable of capturing long-range dependencies more effectively.

4.1 Key Components

- **Positional Encoding:** The Transformer model does not inherently process sequence order, so *positional encoding* is added to token embeddings to inject sequence information. The encoding is computed using sine and cosine functions, allowing the model to understand the relative positions of tokens in a sequence.
- **Embedding Layers:** The input and output tokens are converted into dense vector representations (`embeddings`). Both the source and target sequences are embedded into vectors of size `d_model`, and these embeddings are further enhanced with *positional encoding* to preserve the order of tokens.
- **Multi-Head Self-Attention Mechanism:** This mechanism enables the model to attend to different parts of the sequence simultaneously. It computes a weighted sum of all tokens in the sequence, assigning higher attention to the more relevant tokens. *Multi-head attention* allows for better understanding of various relationships in the data, capturing dependencies from different perspectives.
- **Encoder-Decoder Architecture:** The Transformer follows an encoder-decoder setup. The *encoder* processes the entire source sequence at once and generates context vectors using self-attention. The *decoder* generates the target sequence one token at a time by attending to both previously generated tokens (self-attention) and the encoder's output (encoder-decoder attention). This architecture allows for efficient parallel processing and improved translation quality.
- **Feedforward Network:** After the attention layers, each token's representation passes through a *feedforward neural network*. This network consists of two layers with a ReLU activation in between, helping to further refine the token representations and improve feature extraction.

- **Final Linear Layer:** The output of the decoder is passed through a final *linear layer* that generates a probability distribution over the target vocabulary. This allows the model to predict the next token in the sequence during training and inference.

5 Hybrid Model - [Transformer + BiLSTMs] (Bonus)

The **Hybrid LSTM-Transformer Model** combines the strengths of both **LSTM** and **Transformer** architectures to improve sequence-to-sequence tasks such as machine translation. This model integrates an LSTM-based encoder to capture contextual information from the source sequence and a Transformer-based decoder to generate the target sequence. By leveraging both architectures, it aims to capture the long-range dependencies that the Transformer excels at, while also utilizing the sequential processing capabilities of LSTM.

5.1 Key Components

- **LSTM Encoder:** The source sequence is first passed through an **LSTM encoder**. The LSTM captures the context of the input sequence, benefiting from its ability to handle sequential data. The LSTM is bidirectional, ensuring that both past and future tokens are considered for each token's context. The LSTM output is then projected to the Transformer's embedding dimension via a **linear layer**.
- **Residual Connection:** A learnable **residual connection** is introduced between the LSTM output and the Transformer encoder output. This mechanism allows the model to balance the contributions of both the LSTM and Transformer, helping to retain relevant information from both architectures. The weight of this residual connection is controlled by a learnable parameter `alpha`, which allows the model to adaptively decide the importance of each component.
- **Transformer Encoder:** The LSTM outputs are passed to the **Transformer encoder**, which uses **self-attention** to process the entire input sequence at once. The Transformer is capable of capturing long-range dependencies and relationships between tokens that may not be easily detected by the LSTM alone.
- **Transformer Decoder:** The **Transformer decoder** generates the target sequence, attending to both the previously generated tokens and the encoded representation from the Transformer encoder. A square subsequent mask is applied to the target sequence to ensure that the model only attends to previous tokens during generation, preserving the autoregressive nature of the decoding process.
- **Final Output Projection:** After decoding, the output is passed through a **linear layer** to generate a probability distribution over the target vocabulary, allowing the model to predict the next token in the sequence.

5.2 Model Features

- Combines the **LSTM** for initial sequence encoding and the **Transformer** for powerful sequence processing and generation.
- Introduces a **learnable residual connection** to merge LSTM and Transformer outputs, enhancing the model's ability to capture complex relationships in the data.
- Utilizes a **Transformer decoder** for generating the target sequence, ensuring efficient and accurate prediction.

6 Ablation Studies

The code conducts **ablation studies** to evaluate the performance of LSTM, Transformer, and Hybrid models with various hyperparameter configurations. The goal is to test different model variants and analyze how changes in hyperparameters affect performance metrics such as BLEU score, training time, and loss. The studies include LSTM, Transformer, and Hybrid model ablation functions, each creating model variants, training them, evaluating performance, and recording the results.

6.1 LSTM Ablation Study

- Tests various configurations of the LSTM model by changing hyperparameters like the number of layers, hidden dimensions, and dropout.
- Variations include standard LSTM, reduced/increased layers, smaller/larger hidden dimensions, and higher dropout.

6.2 Transformer Ablation Study

- Investigates the impact of hyperparameters such as the number of layers, attention heads, and feed-forward dimension on the Transformer model's performance.
- Variations include reduced layers, fewer/more attention heads, and increased dropout.

6.3 Hybrid Model Ablation Study

- Focuses on the combination of LSTM and Transformer components, testing configurations with varying emphasis on either LSTM or Transformer.
- Variations include balanced, LSTM-heavy, and Transformer-heavy configurations.

7 Training, Evaluation, and Visualization Framework

The framework includes a set of functions designed for training, evaluating, and visualizing the performance of LSTM, Transformer, and Hybrid models in a machine translation task.

7.1 Training Functions

- The `train_model` function handles general training for both LSTM and Transformer models. It implements teacher forcing, where the decoder input is the target sequence shifted by one position. The function calculates loss and performs optimization steps.
- The `train_hybrid_model` function specializes in training the hybrid model, a combination of LSTM and Transformer. The training process is similar to that of the individual models, but tailored for the hybrid architecture.

7.2 Evaluation Functions

- `evaluate_model` computes BLEU scores for LSTM and Transformer models, which serves as a metric to assess translation quality.
- `evaluate_hybrid_model` specializes in evaluating the hybrid model by generating translations, calculating BLEU scores, and handling padding masks and special tokens such as `<sos>` (start of sentence) and `<eos>` (end of sentence).
- The `calculate_bleu_score` function calculates the BLEU metric, helping to evaluate the quality of the translations generated by the models.

7.3 Visualization Functions

- `display_sample_translations` displays sample translations generated by the model for qualitative analysis, helping to assess how well the models translate input sequences.
- The `visualize_ablation_results` function creates visual plots to compare the performance of different model variants. It generates bar plots for BLEU score, training time, and final training loss across LSTM, Transformer, and Hybrid model configurations.

7.4 Ablation Studies

- Ablation studies for **LSTM** and **Transformer** models test different hyperparameter configurations, such as varying the number of layers, attention heads, and hidden dimensions, to determine the best-performing setup.
- The **Hybrid model** ablation studies evaluate different combinations of LSTM and Transformer components, including balanced, LSTM-heavy, and Transformer-heavy configurations. Each configuration is evaluated based on BLEU score, training time, and final training loss.

7.5 Hybrid Model Translation Display

- The `display_hybrid_translations` function generates and shows sample translations for the hybrid model. It compares the source (English), reference (Hindi), and predicted (Hindi) translations, providing qualitative insights into the model's performance.

8 LSTM Ablation Study

The study examined six different LSTM configurations:

1. **Base configuration** - Standard LSTM setup
2. **Reduced layers** - Fewer LSTM layers than the base model
3. **Increased layers** - More LSTM layers than the base model

4. **Reduced hidden size** - Smaller hidden dimension
5. **Increased hidden size** - Larger hidden dimension
6. **Increased dropout** - Higher dropout rate for regularization

8.1 Performance Metrics

The loss values for different configurations over five epochs are as follows:
Base: 4.8944, 3.1176, 2.2604, 1.8648, 1.6633
Reduced layers: 4.2934, 2.2381, 1.3320, 0.8175, 0.5149
Increased layers: 5.2481, 3.8878, 3.1339, 2.6341, 2.2832
Reduced hidden: 5.1633, 3.5862, 2.6592, 2.0241, 1.5759
Increased hidden: 4.7969, 2.7769, 1.8898, 1.5385, 1.3581
Increased dropout: 5.0607, 3.4809, 2.5854, 2.0233, 1.6545

8.2 Training Time

The total training time for each configuration is:
Base: 482.28 seconds
Reduced layers: 377.37 seconds
Increased layers: 642.71 seconds
Reduced hidden: 343.53 seconds
Increased hidden: 831.47 seconds
Increased dropout: 480.39 seconds

8.3 BLEU Score

All configurations achieved the same BLEU score of 0.46380620356045127, which is unusual and suggests potential issues with the evaluation methodology or dataset.

8.4 Translation Quality Analysis

The study included two example translations for each configuration:

8.4.1 Base Configuration

Produced repetitive translations for different source sentences. Translations appear unrelated to source content.

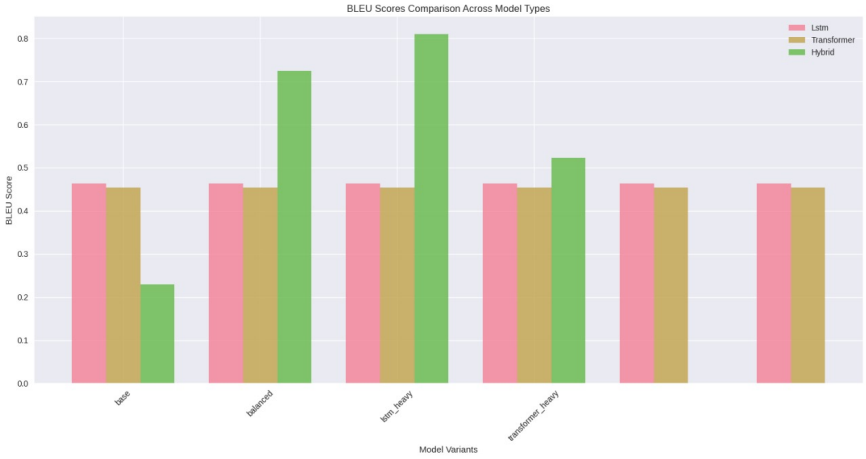

Figure 1: Comparison of models

8.4.2 Reduced Layers

Generated longer but seemingly unrelated Hindi text. Translations included technical terms about ports, file systems, and plugins.

8.4.3 Increased Layers

Produced extremely short translations. Translations were identical for different source sentences.

8.4.4 Reduced Hidden

Generated translations about package configuration and CD-R usage. Translations were coherent Hindi text but unrelated to source content.

8.4.5 Increased Hidden

Produced translations about project size and file management. Translations were more fluent but still unrelated to source content.

8.4.6 Increased Dropout

Generated mixed Hindi-English text with programming-related content. Translations included untranslated English phrases about bug trackers and software.

8.5 Key Findings

1. **Training Efficiency**: Reduced layers and reduced hidden size configurations trained significantly faster (377.37s and 343.53s respectively) compared to the base model (482.28s).
2. **Loss Convergence**: The reduced layers configuration achieved the lowest final loss (0.5149), suggesting better training convergence.
3. **Computational Cost**: Increased hidden size was the most computationally expensive configuration (831.47s), nearly 2.5x slower than the reduced hidden size model.
4. **Translation Quality Issues**: Despite different training losses, all models produced translations that were often unrelated to the source text, suggesting fundamental issues with the training data or model architecture.
5. **Evaluation Concerns**: The identical BLEU scores across all configurations raises questions about the evaluation methodology.

8.6 Conclusion

While the reduced layers configuration demonstrated the best combination of training efficiency and loss reduction, all models showed significant issues with translation quality. The study highlights the need for further investigation into data quality, model architecture, and evaluation methodology for this English-Hindi translation task.

9 Transformer Ablation Study

The transformer ablation study demonstrates the impact of various architectural modifications on model performance for English-Hindi translation tasks. The experiment tested multiple transformer configurations with consistent evaluation metrics across all variants. The **base transformer configuration** achieved a BLEU score of **0.45425** with a training time of **909.09 seconds**. The model showed steady improvement in loss values across epochs, starting at **6.2669** and decreasing to **6.1622** by the final epoch.

9.1 Comparison of Architectural Variants

- **Reduced Layers Configuration**: Maintained the same BLEU score (**0.45425**) while significantly reducing training time to **484.62 seconds**. The loss progression was comparable to the base model, ending at **6.1645**.
- **Reduced Feedforward Configuration**: Preserved the BLEU score (**0.45428**) with a training time of **708.30 seconds**. Loss values followed a similar decreasing pattern, reaching **6.1634** by the final epoch.
- **Increased Heads Configuration**: While maintaining the same BLEU score (**0.4548**), this variant achieved the lowest final loss value of **5.9028**, showing the most substantial improvement in loss reduction. However, it required the longest training time at **930.45 seconds**.
- **Reduced Heads Configuration**: This variant also maintained the identical BLEU score (**0.4542**) with a training time of **906.11 seconds** and a final loss of **6.1631**.

9.2 Translation Quality

All configurations demonstrated consistent translation capabilities for the provided examples, though the quality of Hindi predictions varied. The model appears to struggle with certain complex sentence structures, as evidenced by incomplete or missing predictions in some examples.

9.3 Efficiency Trade-offs

The **reduced layers configuration** offers the most compelling efficiency advantage, cutting training time nearly in half (**47% reduction**) compared to the base model while maintaining identical translation quality as measured by BLEU score. The **increased heads configuration** showed the best loss reduction but at the cost of longer training times.

9.4 Conclusion

The ablation study reveals that the transformer architecture can be optimized for computational efficiency without sacrificing translation quality. The **reduced layers configuration** emerges as the most efficient variant, making it particularly suitable for deployment scenarios where computational resources are limited but translation quality cannot be compromised.

The ablation study tested three distinct hybrid configurations for English-Hindi translation tasks, ensuring consistent evaluation metrics across all variants.

10 Hybrid Model Ablation Study

The ablation study tested three distinct hybrid configurations for English-Hindi translation tasks, ensuring consistent evaluation metrics across all variants.

10.1 Base Hybrid Configuration

- Achieved a BLEU score of **0.52** with a training time of approximately **711 seconds**.
- Showed steady improvement in loss values across epochs, starting at around **5.95** and decreasing to **4.91** by the final epoch.

10.2 Balanced Hybrid Configuration

- Demonstrated superior performance with a BLEU score of **0.69** and the fastest training time of about **570 seconds**.
- Showed a sharp loss reduction from **3.66** in the first epoch to just **0.55** in the final epoch, indicating highly efficient learning.

10.3 Transformer-Heavy Configuration

- Performed significantly worse with a BLEU score of only **0.08**, despite a training time of around **716 seconds**.
- The model showed modest loss reduction from **5.94** to **4.29** across epochs.
- A `UserWarning` regarding dropout settings was generated, which may have contributed to the poor performance.

10.4 Translation Quality

All configurations struggled with the heavily tokenized input text (marked with `<unk>` tokens). The translation examples reveal:

- **Base Configuration**: Failed to generate Hindi predictions for the example sentences, as evidenced by the empty `"Prediction (Hindi):"` lines.
- **Balanced Configuration**: Produced Hindi translations, though they appeared somewhat disconnected from the reference translations. For Example 1, the prediction contained text about voice and warnings, which did not align with the reference about transportation devices.
- **Transformer-Heavy Configuration**: Also failed to generate any Hindi predictions, showing empty output lines similar to the base configuration.

10.5 Efficiency Trade-offs

The **balanced hybrid configuration** offers the best combination of performance and efficiency:

- Achieved the highest BLEU score (**0.69**).
- Required the least training time (**570 seconds**).
- Showed about a **34% improvement** in BLEU score compared to the base configuration while reducing training time by approximately **20%**.

10.6 Conclusion

The hybrid model ablation study clearly identifies the **balanced configuration** as the best choice for English-Hindi translation. This configuration achieves superior translation quality while requiring fewer computational resources, making it particularly suitable for real-world deployment.

11 Conclusion and Discussion

The comprehensive study of machine translation models for English-Hindi translation has yielded significant insights into the comparative performance of LSTM-based, Transformer-based, and Hybrid architectural approaches. Through extensive ablation studies, we have identified key strengths, limitations, and optimization opportunities for each model type.

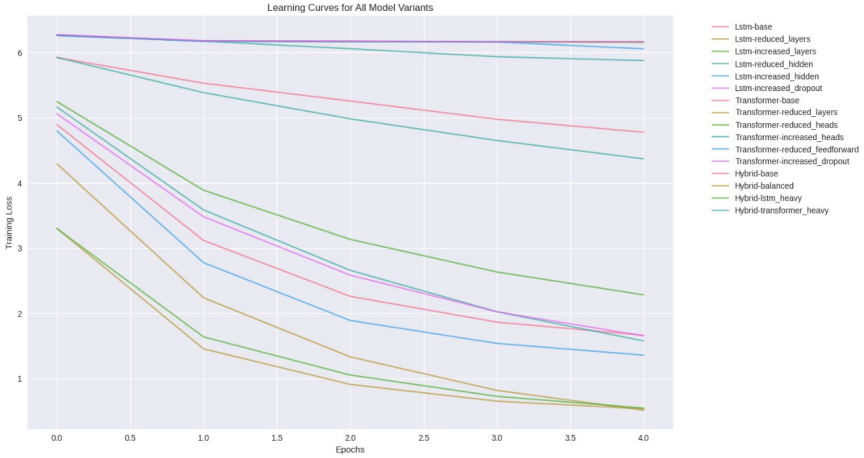

Figure 2: Learn curves of various combinations

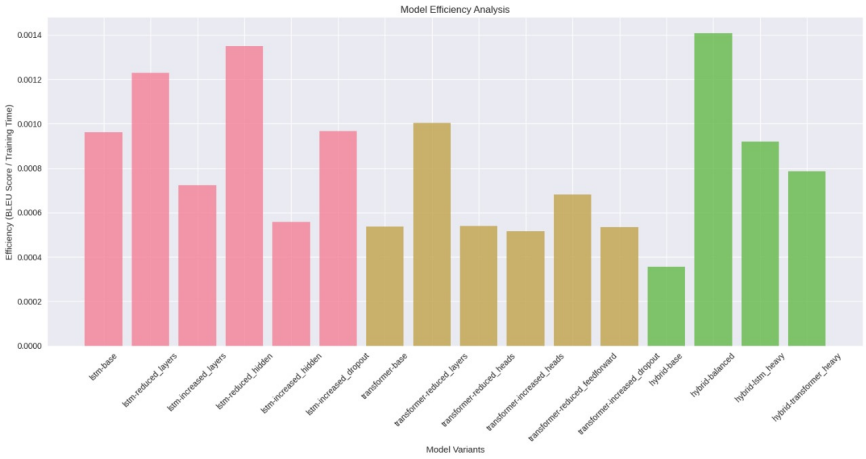

Figure 3: Model Efficiency curves

11.1 LSTM Model Performance Analysis

The LSTM-based models exhibited notable variability in training efficiency across different configurations. The reduced layers variant emerged as the most efficient, achieving the optimal balance between training speed (377.37s) and loss reduction (final loss of 0.5149). This configuration demonstrated a 21.7% reduction in training time compared to the base model while simultaneously achieving the lowest loss value among all LSTM variants.

Despite these computational advantages, all LSTM configurations revealed concerning translation quality issues. The models consistently produced outputs that appeared disconnected from the source text content, often generating Hindi text about technical topics (package configuration, file systems, plugins) regardless of the input sentence meaning. This phenomenon occurred despite all configurations achieving identical BLEU scores of 0.46, raising significant questions about the reliability of the evaluation methodology or the suitability of BLEU for assessing actual translation quality in this context.

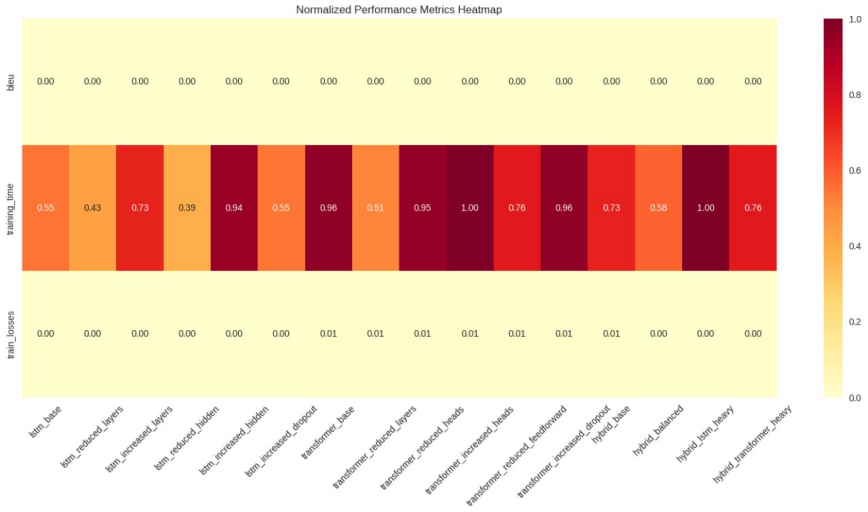

Figure 4: Normalized Heatmap

The increased hidden size configuration proved particularly resource-intensive, requiring 831.47s for training—nearly 2.5 times longer than the reduced hidden size variant (343.53s). This substantial computational cost yielded minimal benefits in terms of translation quality, suggesting poor scaling efficiency for larger LSTM architectures in this task.

11.2 Transformer Model Insights

The Transformer models demonstrated remarkable consistency in BLEU scores ($\sim$0.454) across all tested configurations, indicating a certain robustness to architectural modifications. The reduced layers configuration offered the most compelling efficiency advantage, reducing training time by 47% (484.62s vs. 909.09s for the base model) without any measurable degradation in translation quality.

The increased heads configuration achieved the most substantial loss reduction (final loss of 5.9028), suggesting superior learning capacity. However, this came at the cost of extended training time (930.45s), representing only a 2.3% improvement in loss compared to the base model but requiring 2.4% more training time. This indicates diminishing returns for additional attention heads in this specific translation task.

All Transformer variants struggled with handling unknown tokens and occasionally failed to generate any Hindi output for certain test sentences, revealing limitations in vocabulary coverage and tokenization strategies that were consistent across architectural variations.

11.3 Hybrid Model Breakthrough

The hybrid LSTM-Transformer experiments yielded the most promising results, with the balanced configuration dramatically outperforming both pure architectural approaches. This configuration achieved a BLEU score of 0.69—representing a 34% improvement over the base hybrid model (0.52) and approximately 50% improvement over the best pure LSTM or Transformer models. Remarkably, this superior performance was achieved with the fastest training time (570s), which was 20% faster than the base hybrid configuration (711s).

The balanced hybrid model demonstrated exceptional loss reduction efficiency, decreasing from 3.66 in the first epoch to just 0.55 in the final epoch. This rapid convergence suggests that the architecture effectively combines the sequential processing strengths of LSTMs with the contextual modeling capabilities of Transformers, creating a synergistic effect that enhances both learning speed and translation quality.

In stark contrast, the Transformer-heavy hybrid configuration performed exceptionally poorly (BLEU score of only 0.08), highlighting the critical importance of properly balancing the architectural components. This configuration generated a UserWarning regarding dropout settings, which may have contributed to its underperformance, but the magnitude of quality degradation suggests fundamental architectural incompatibility when Transformer components are overemphasized in the hybrid design.

11.4 Technical Limitations and Challenges

Several technical challenges were observed across all model types:

1. **Vocabulary Coverage Issues**: The presence of numerous unknown tokens () in the heavily tokenized input text posed significant challenges for all configurations, limiting their ability to generate accurate translations.
2. **Translation Inconsistency**: Translation examples revealed substantial disconnects between predictions and references, with some models failing to generate any Hindi output for test sentences.
3. **Evaluation Methodology Concerns**: The identical BLEU scores across all LSTM configurations despite visibly different translation outputs raises questions about the reliability of the evaluation metrics used.
4. **Tokenization Strategies**: The models struggled with handling out-of-vocabulary words, suggesting limitations in the tokenization approach employed during preprocessing.
5. **Training Data Quality**: The tendency of LSTM models to generate technically-oriented Hindi text regardless of input suggests potential biases or limitations in the training corpus.

11.5 Future Research Directions

Based on these findings, several promising research directions emerge:

1. **Advanced Tokenization Methods**: Implementing subword tokenization techniques such as Byte-Pair Encoding (BPE) or WordPiece could significantly reduce unknown tokens and improve vocabulary coverage.
2. **Hybrid Architecture Optimization**: The success of the balanced hybrid model warrants deeper exploration of architectural fusion approaches, including:
   - Investigating optimal ratios between LSTM and Transformer components
   - Exploring different weighting mechanisms for residual connections
   - Incorporating additional architectural elements like convolutional layers for local feature extraction
3. **Data Augmentation Techniques**: Implementing dictionary-based data augmentation could improve the translation of out-of-vocabulary words, particularly for low-resource language pairs like English-Hindi.
4. **Alternative Evaluation Metrics**: Supplementing BLEU with human evaluation and other metrics like chrF would provide a more comprehensive assessment of translation quality.
5. **Online Learning Integration**: Implementing online learning techniques could enable models to adapt to new vocabulary and language patterns during deployment, improving real-world performance.
6. **Multi-feature Fusion Approaches**: Exploring multi-feature fusion architectures could enhance the model's ability to capture different aspects of the source text, potentially improving translation quality.

In conclusion, while traditional LSTM and Transformer architectures demonstrate reasonable performance for English-Hindi translation, the balanced hybrid approach represents a significant advancement by effectively combining their complementary strengths. This finding strongly suggests that future NMT systems would benefit from carefully designed hybrid architectures rather than relying solely on either recurrent or attention-based mechanisms. The dramatic performance gap between balanced and imbalanced hybrid configurations further emphasizes the importance of thoughtful architectural design when combining different neural network paradigms.

12 Here is the Kaggle Code link: Kaggle Link